

第 14 章 软件实现与测试

软件实现阶段是将软件设计结果付诸实施的过程。这一阶段的任务包括计算机等设备的购置、设备安装和调试、程序编写与调试等。这个阶段的特点是几个互相联系、互相制约的任务同时展开,因此,必须精心安排、合理组织。软件实现是按实施计划分阶段完成的,每个阶段应写出实施进展报告。

软件实现时选用的程序设计方法、程序设计语言和程序设计风格会对系统的可靠性、可维护性和可复用性产生很大的影响。

软件测试是软件开发生命周期中的重要阶段之一,旨在检查和验证软件系统的各个方面,以确保其质量、可靠性和功能完整性。软件测试阶段是确保软件质量和稳定性的关键阶段,可以帮助发现并修复潜在问题,从而为用户提供可靠的产品。

软件测试是软件质量保证的主要手段之一,其目的就是在软件正式运行前,尽可能多地找出软件中潜在的各种错误和缺陷。

14.1 软件实现概述

在信息系统建设中,软件实现的主要任务是进行程序设计,就是把软件设计结果翻译成用某种程序设计语言书写的程序。作为软件生命周期过程的一个阶段,程序设计是对软件设计的进一步具体化。

14.1.1 程序设计方法

程序设计方法是软件工程方法学的主要内容之一,主要有结构化程序设计、面向对象程序设计和可视化程序设计。

1. 结构化程序设计

结构化程序设计采用自顶向下、逐步求精的设计方法和单入口、单出口的控制结构。在设计一个模块的实现算法时,先考虑整体后考虑局部,先抽象后具体,通过逐步细化,最后得到详细的实现算法。单入口、单出口的控制结构,使程序的静态结构和动态执行过程一致,具有良好的结构,增强了程序的可读性。

1966 年,Bohm 和 Jacopini 证明了任何单入口、单出口、没有死循环的程序都能用三种基本的控制结构来构造,这三种基本的控制结构是顺序结构、IF-THEN-ELSE 型分支结构(选择结构)和 DO-WHILE 型循环结构。如果程序设计中只允许使用这三种基本的控制结构,则称为经典的结构化方法;如果还允许使用 DO-CASE 型多分支结构和 DO-UNTIL 型循环结构,则称为扩展的结构化方法;如果再加上允许使用 LEAVE(或 BREAK)结构,则称为修正的结构化方法。

2. 面向对象程序设计

面向对象程序设计（OOP）是面向对象方法学从诞生、发展到走向成熟的第一片领地，也是使面向对象方法最终落实的重要阶段。在一个比较理想的 OOP 程序中，问题域中有哪些值得注意的事物，程序中就有哪些对象；问题域中的事务之间是什么关系，程序中的对象之间就具有什么关系。在面向对象分析（OOA）和面向对象设计（OOD）理论出现之前，程序员要写一个面向对象的程序，首先要学会运用面向对象方法来认识问题域，所以，OOP 被看作是一门比较高深的技术。现在，在“OOA → OOD → OOP”的软件工程过程中，OOP 的分工就比较简单了。认识问题域和设计系统成分的工作已经在 OOA 和 OOD 实现，OOP 的工作就是用一种面向对象的程序设计语言把 OOD 模型中的每个成分书写下来。

由于面向对象程序设计语言具有支持类、对象、继承、多态和消息通信等 OO 概念的机制，OOP 可以显著提高软件的可靠性、可维护性和可复用性。

3. 可视化程序设计

目前，程序员可以利用程序设计工具所提供的各种控件，像搭积木一样地构造应用程序的各种界面，这种程序设计方法称为可视化程序设计（Visual Programming, VP）。VP 最大的优点是程序员可以不用编写或只需编写很少的程序代码，就能完成应用程序的设计，从而极大地提高设计人员的工作效率。能进行 VP 的工具很多，比较常用的有 Eclipse、VisualStudio 等。

VP 是一种事件驱动的程序设计方法，其基本概念包括表单、部件、属性、事件和方法等。表单是指进行程序设计时的窗口，程序员主要通过在表单中放置各种部件来布置应用程序的运行界面；部件是指组成程序运行界面的各种构件，例如，命令按钮、复选框、单选框和滚动条等；属性是指部件的性质，它说明部件在程序运行的过程中是如何显示的、部件的大小、显示在何处、是否可见和是否有效等；事件是指对一个部件的操作，例如，用鼠标单击一个命令按钮时，单击鼠标就称为一个事件（Click 事件）；方法是指某个事件发生后要执行的具体操作，也就是事件响应程序或事件处理程序。VP 的主要过程就是在表单中放置各种部件、定义事件的属性和编写事件响应程序等。

14.1.2 程序设计语言与风格

程序设计语言是人机通信的基本工具，其特点必然会影响人的思维和解题方式，会影响人机通信的方式和质量，也会影响其他人阅读和理解程序的难易程度。因此，在进行程序设计之前的一项重要工作，就是选择一种适当的程序设计语言。

良好的程序设计语言能使程序员根据软件设计说明书完成程序设计时困难最少，可以减少所需要的程序测试量，并且可以得出更容易阅读和更容易维护的程序。

1. 程序语言的选择

为了使程序容易测试和维护，以减少软件的总成本，所选用的程序设计语言应该有理想的模块化机制，以及可读性好的控制结构和数据结构；为了便于调试和提高软件可靠性，所选用的程序设计语言应该使编译程序能够尽可能多地发现程序中的错误；为了降低软件开发和维护的成本，所选用的程序设计语言应该有良好的独立编译机制。实际选择程序设计语言时，还必

须同时考虑实用方面的各种限制,例如,系统用户的要求、可以使用的编译程序、可以得到的软件工具、工程规模、程序员的知识、软件可移植性要求和软件的应用领域等。

2. 程序设计风格

良好的程序设计风格可以提高程序的可理解性、可复用性、可扩展性和健壮性,不仅能明显减少维护或扩展的开销,而且有助于在新项目中移植和复用已有的程序代码。

与可理解性相关的良好程序设计风格包括有意义的标识符、详细的注解和程序的视觉组织、清晰规范的数据说明和简单明了的语句构造,以及有效、合理、交互化与可视化的 I/O 设计等。与可复用性相关的良好程序设计风格包括提高功能的内聚、减小功能的规模、保持功能的一致性、将接口与实现分开、尽量不使用全局变量和利用继承机制等。与可扩展性相关的良好程序设计风格包括封装实现策略、利用多态性机制、避免使用多分支语句和精心设计公有服务等。与健壮性相关的良好程序设计风格包括预防用户的错误操作、检查参数的合法性、不要预先确定限制条件和先测试后优化等。

14.1.3 编码规范

应用编码规范对于软件本身和软件开发人员而言尤为重要,有以下几个原因:

(1) 好的编码规范可以尽可能地减少一个软件的维护成本,并且几乎没有任何一个软件在其整个生命周期中均由最初的开发人员来维护。

(2) 好的编码规范可以改善软件的可读性,可以让开发人员尽快并彻底理解新的代码。

(3) 好的编码规范可以最大限度地提高团队开发的合作效率。

(4) 长期的规范性编码还可以让开发人员养成好的编码习惯,甚至锻炼出更加严谨的思维方式。

编码规范一般会从命名规范、注释原则、编程风格规则和版本管理规则这四个方面来制定对应的要求。

常见的编码规范有命名规范、格式化规范、注释规范、函数长度规范、错误处理规范、冗余代码规范和通用开发模式规范等。

14.1.4 代码生成

在实际软件开发过程中,关注的重点是软件业务功能的开发及怎样保证业务流程的正确性。对于重复性的代码编写会占据大量的时间和精力,比如用户认证、权限管理等模块中的结构,用户、角色、权限等模块都有类似的结构。这部分代码往往有固定的规律,可以使用代码生成器来自动生成,减少手工的重复劳动,同时能把更多的精力用在关注更核心的业务逻辑以及软件的优化上面。

代码生成的实现步骤如下:

(1) 准备模板:生成文件的模板文件。

(2) 准备数据:生成文件所需要的关键数据。

(3) 定义合成机制:使用数据置换模板中的占位符,即生成新的文件的机制。

- (4) 遍历所有的数据文件，在遍历数据的同时加载所有的模板文件。
- (5) 模板 + 数据 = 生成的代码文件。

14.1.5 代码重用

软件重用可以分为三个层次，代码重用、设计结果重用和分析结果重用。其中，代码重用是对现有（已经写好的）代码进行重用，这些代码来自外部资源或过往项目，并用之开发新软件。程序员这样做是为了实现相同或相似的功能。然而，只有高质量的代码（无错或不复杂）才能被重用。

开发快速、可靠和安全的软件始终需要大量编程技能和知识，因此，开发人员需要先分析应用程序的条件和要求，然后再进行代码重用。但是代码重用并不适合所有软件开发，因此开发人员有必要先评估重用条件，然后再进行代码重用。

常用的实现代码重用的方法有：

- (1) 使用函数：将代码抽象成函数，这样只需在多个地方调用该函数即可。
- (2) 使用对象：在软件中，将复杂的代码封装成一个个对象，这样代码就可以被重复使用。
- (3) 使用模块：将代码抽象成模块，这样使用者只需调用模块即可实现相同的功能，提高了程序的可重用性。
- (4) 使用类：利用面向对象的思想，将代码封装成类，通过继承和多态可以实现高度复用，把代码封装成复用性高的单元。
- (5) 利用算法：采用解构和分解组合算法可以减少代码重复使用的损耗。

14.2 软件测试概述

随着计算机技术的迅速发展和越来越广泛深入地应用于国民经济和社会生活的各个方面，随着软件系统的规模和复杂性与日俱增，软件的生产成本和软件中存在的缺陷和故障造成的各类损失也大大增加，甚至会带来灾难性的后果。软件质量问题已成为所有使用软件和开发软件的人关注的焦点。

测试是最有效的排除和防止软件缺陷与故障的手段，并由此促进了软件测试理论与技术实践的快速发展，新的测试理论、测试方法、测试技术手段在不断涌出，软件测试机构和组织也在迅速产生和发展，由此软件测试技术职业也同步完善和健全起来。

在开发大型软件系统过程中，我们力求在每个阶段结束之前通过严格的技术审查，尽可能早地发现并纠正差错。如果在软件投入生产性运行之前，没有发现并纠正软件中的大部分差错，则这些差错终究会在生产过程中暴露出来，那时不仅改正这些差错的代价更高，而且往往会造成很恶劣的后果。测试的目的就是在软件投入生产运行之前，尽可能多地发现错误，而且无论怎样强调软件测试的重要性和它对软件可靠性的影响都是不过分的。

14.2.1 软件测试的概念

软件测试是一项技术型工作，同时，它也涉及一些人的心理因素。测试执行不佳的主要原

因之一是大多数测试人员开始误解“测试”一词的定义，他们认为：

- (1) 软件测试的目标是软件能够正确地完成预期的功能。
- (2) 软件测试就是证明软件不存在错误的过程。
- (3) 软件测试就是建立“软件做了其应该做的”信息的过程。

这些想法都是本末倒置的。软件测试应当是寻找软件的缺陷及漏洞，通过测试提高程序的价值、可靠性或质量。因此软件测试不应只是正向思维，也应该要逆向思维，甚至要发散思维。在测试开始时，测试人员应该假设程序中存在隐藏的错误，然后通过测试找到尽可能多的错误。软件测试就是通过不断地发现问题、解决问题来提高软件的质量。当然，软件测试的标准定义是软件测试使用手动或自动方式来运行或测试系统，其目的是测试其是否满足所需的要求或明确预期结果与实际结果之间的差异。

IEEE 对软件测试的定义是：使用人工或自动的手段来运行或测量软件系统的过程，目的是检验软件系统是否满足规定的需求，并找出与预期结果之间的差异。软件测试的发展趋势如下：

- (1) 测试工作将进一步前移。软件测试不仅是单元测试、集成测试、系统测试和验收测试，还有对需求的精确性和完整性的测试，以及对系统设计的测试。
- (2) 软件架构师、开发工程师、质量保证人员、测试工程师的工作将更好地融合。
- (3) 测试职业将得到更充分的尊重。
- (4) 设置独立的软件测试部门将成为未来软件公司的共识。
- (5) 测试外包服务将快速增长，和软件开发外包一样，软件测试外包将呈现全球化的趋势。

14.2.2 软件测试的基本问题

软件测试的基本问题是验证功能的实现和可用性，检查程序的错误，最终目的是提高用户体验，以较少的人力、物力和时间找出软件中潜在的各种错误和缺陷，通过修正各种错误和缺陷提高软件质量，回避软件发布后由于潜在的软件缺陷和错误造成的隐患所带来的商业风险。

同时，测试是以评价一个程序或者系统属性为目标的活动。测试是对软件质量的度量与评估，以验证软件的质量满足用户需求的程度，为用户选择和接收软件提供有力依据。

14.2.3 软件测试的对象及目的

软件测试的对象包括：源程序 / 目标代码；各开发阶段的文档（需求规格说明、概要设计说明、详细设计说明及其他相关文档）配置数据。

软件测试的目的包括：

- (1) 从用户角度看的目的：通过软件测试发现隐藏的错误和缺陷，考虑是否可以接受该产品。
- (2) 从开发者角度看的目的：表明软件产品不存在错误，验证软件实现了所有用户的要求。
- (3) 从测试人员角度看的目的：发现错误，预测错误，提供软件可靠性错误，对软件做出评价。

14.2.4 软件测试的原则

软件测试的原则包括以下几个方面：

(1) 测试用例的一个必要部分是预期输出和结果的定义。用例必须包括程序输入数据的部分描述和输入数据下程序正确输出的准确描述。

(2) 程序员应该避免测试他们自己的程序。因为程序员很难以一种“破坏性”的视角来审查自己的程序，就如文档作者在编辑或校对自己的文档时一般无法找出错误的道理是一样的，程序员也存在同样的问题。

(3) 编写软件的组织不应当测试自己编写的软件。这类似于先前的一个论点，即软件项目或编程组织是一个与单个程序员有相似心理的有机组织，因此需要由更客观和独立的第三方进行测试。

(4) 应当具有足够的耐心及细心检查每一个测试结果。

(5) 测试用例不仅应基于有效和预期的输入，而且还应基于通常不会执行的无效和意外输入。当程序以新的或意想不到的方式运行时，软件产品中突然暴露出来的许多问题都会被发现。因此，有必要对不可预见的和无效的输入情况进行测试。

(6) 应避免重新设计测试用例。当软件需要重新测试时，重新设计用例需要投入大量的工作，所以测试用例的复用性及保留测试用例是有必要的。

(7) 计划测试工作不应默认假定不会发现错误，这是使用了不正确的软件测试定义的一个迹象。

(8) 测试不要穷举。穷举是指测试时考虑所有可能的输入值，若在测试过程中穷举所有数据组合需要耗费大量的时间。测试人员需要考虑程序逻辑的全覆盖，并确保程序中使用的所有条件都是可能的，并且以最少的用例达到最大的覆盖率。

(9) 软件测试要尽早执行。软件生命周期包括软件开发、测试、维护的整个流程，主要包括需求阶段、设计阶段、编码阶段、单元测试阶段、验收阶段和维修阶段。实践证明，在大多数情况下，需求分析阶段会出现缺陷，但维修成本很低，因此软件测试应尽快进行并贯穿整个软件生命周期。

(10) 软件测试应追溯到需求。软件测试的目标是发现问题。当然，满足用户的需求更为重要。

14.3 软件测试方法

按被测程序是否可见，可将软件测试方法分为黑盒测试（Black-box Testing）、白盒测试（White-box Testing）和灰盒测试（Gray-box Testing）；按是否需要执行被测程序，可将软件测试方法分为静态测试（Static Testing）和动态测试（Dynamic Testing）。

14.3.1 按照被测程序是否可见分类

1. 黑盒测试

黑盒测试又叫功能测试、数据驱动测试或基于需求规格说明书的功能测试。该类测试注重于测试软件的功能性需求。在测试中，把程序看作一个不能打开的黑盒子，在完全不考虑程序

内部结构和内部特性的情况下,在程序接口进行测试,它只检查程序功能是否按照需求规格说明书的规定正常使用,程序是否能适当地接收输入数据而产生正确的输出信息。黑盒测试着眼于程序外部结构,不考虑内部逻辑结构,主要针对软件界面和软件功能进行测试。

黑盒测试是从用户的角度,从输入数据与输出数据的对应关系出发进行测试的。很明显,如果外部特性本身设计有问题或规格说明的规定有误,用黑盒测试方法是发现不了的。

采用这种测试方法,测试工程师把测试对象看作一个黑盒子,完全不考虑程序内部的逻辑结构和内部特性,只依据程序的需求规格说明书检查程序的功能是否符合它的功能说明。测试工程师无须了解程序代码的内部构造,完全模拟软件产品的最终用户使用该软件,检查软件产品是否达到了用户的需求。黑盒测试方法能更好、更真实地从用户角度来考察被测系统的功能性需求实现情况。在软件测试的各个阶段,如单元测试、集成测试、系统测试及验收测试等阶段中,黑盒测试都发挥着重要作用,尤其在系统测试和确认测试中,其作用是其他测试方法无法取代的。

黑盒测试方法着重测试软件的功能需求,是在程序接口上进行的测试,主要是为了发现以下错误:

- (1) 是否有功能错误,是否有功能遗漏。
- (2) 是否能够正确地接收输入数据并产生正确的输出结果。
- (3) 是否有数据结构错误或外部信息访问错误。
- (4) 是否有程序初始化和终止方面的错误。

从理论上讲,黑盒测试只有采用穷举输入测试,把所有可能的输入都作为测试情况来考虑,才能查出程序中所有的错误。实际上测试情况有无穷多个,人们不仅要测试所有合法的输入,而且还要对那些不合法但可能的输入进行测试。这样看来,完全测试是不可能的,所以我们要进行有针对性的测试,通过制定测试案例指导测试的实施,保证软件测试有组织、按步骤、有计划地进行。黑盒测试行为必须能够加以量化,才能真正保证软件质量,而测试用例就是将测试行为具体量化的方法之一。具体的黑盒测试用例设计方法包括等价类划分法、边界值分析法、错误推测法、因果图法、判定表驱动法、正交试验设计法、功能图法和场景法等。

2. 白盒测试

白盒测试又称为结构测试、透明盒测试、逻辑驱动测试或基于代码的测试。白盒测试是一种测试用例设计方法,“盒子”指的是被测试的软件,“白盒”指的是盒子是可视的,即清楚盒子内部的东西以及里面是如何运作的。白盒测试法全面了解程序内部逻辑结构、对所有逻辑路径进行测试。白盒测试法是穷举路径测试。在使用这一方法时,测试者必须检查程序的内部结构,从检查程序的逻辑着手,得出测试数据。贯穿程序的独立路径数是天文数字。

白盒测试的测试方法有代码检查法、静态结构分析法、静态质量度量法、逻辑覆盖法、基本路径测试法、域测试、符号测试、路径覆盖和程序变异。

白盒测试法的覆盖标准有逻辑覆盖、循环覆盖和基本路径测试。其中,逻辑覆盖包括语句覆盖、判定覆盖、条件覆盖、判定/条件覆盖、条件组合覆盖和修改条件判断覆盖。这6种覆

盖标准发现错误的能力呈由弱到强变化。

- (1) 语句覆盖：每条语句至少执行一次。
- (2) 判定覆盖：每个判定的每个分支至少执行一次。
- (3) 条件覆盖：每个判定的每个条件应取到各种可能的值。
- (4) 判定 / 条件覆盖：同时满足判定覆盖条件覆盖。
- (5) 条件组合覆盖：每个判定中各条件的每一种组合至少出现一次。

(6) 修改条件判断覆盖：每一个判断的所有可能结果都出现过、每一个判断中所有条件的所有可能结果都出现过、每一个进入点及结束点都执行过、判断中每一个条件都可以独立影响判断的结果。

白盒测试的目的是通过检查软件内部的逻辑结构，对软件中的逻辑路径进行覆盖测试。在程序不同地方设立检查点，检查程序的状态，以确定实际运行状态与预期状态是否一致。

白盒测试需要遵循的原则如下：

- (1) 一个模块中的所有独立路径至少被测试一次。
- (2) 所有逻辑值均需测试 true 和 false 两种情况。
- (3) 检查程序的内部数据结构，保证其结构的有效性。
- (4) 在取值的上、下边界及可操作范围内运行所有循环。

白盒测试“三步法”具体如下：

- 第1步，根据代码的功能，人工设计测试用例进行基本功能测试。
- 第2步，统计白盒覆盖率，为未覆盖的白盒单位设计测试用例，实现完整的白盒覆盖，比较理想的覆盖率是实现100%的语句、条件、分支、路径覆盖。
- 第3步，自动生成大量的测试用例，捕捉“程序员未处理某些特殊输入”形成的错误。

第1步的测试用例通常是比较容易获得的，因为文档会规定程序的基本功能，没有文档的，程序员在编程时也要想清楚程序的功能，这些基本功能就是基本测试用例。

第2步是在第1步的基础上，检查未覆盖的白盒单位，由于未覆盖的逻辑单位通常对应未测试的等价类，因此第2步可以找出第1步所遗漏的测试用例。

第3步用自动动态测试弥补第2步的固有缺陷。

“三步法”尽量避免重复工作，将白盒测试方法和黑盒测试方法相结合，人工测试方法和自动测试方法相补充，如果第2步的覆盖率比较理想，那么基本上可以保证找出所有等价类。在开发过程允许的限度内，“三步法”已接近测试极限。

白盒测试主要用在具有高可靠性要求的软件领域，例如，军工软件、航天航空软件、工业控制软件等。在选购白盒测试工具时应当主要考虑对开发语言的支持、代码覆盖的深度、嵌入式软件的测试和测试的可视化等。

3. 灰盒测试

灰盒测试是介于白盒测试与黑盒测试之间的一种测试，灰盒测试多用于集成测试阶段，不仅关注输出、输入的正确性，同时也关注程序内部的情况。灰盒测试不像白盒那样详细、完整，但又比黑盒测试更关注程序的内部逻辑，常常是通过一些表征性的现象、事件、标志来判断内

部的运行状态。

灰盒测试由方法和工具组成,这些方法和工具取材于应用程序的内部知识和与之交互的环境,能够用于黑盒测试以增强测试效率、错误发现和错误分析的效率。

然而,灰盒测试相对白盒测试更加难以发现并解决潜在问题,尤其在一个单一的应用中,白盒测试的内部细节可以完全掌握。灰盒测试结合了白盒测试和黑盒测试的要素。它考虑了用户端、特定的系统知识和操作环境。它在系统组件的协同性环境中评价应用软件的设计。灰盒测试由方法和工具组成,这些方法和工具取材于应用程序的内部知识和与之交互的环境,能够用于黑盒测试以增强测试效率、错误发现和错误分析的效率。灰盒测试涉及输入和输出,但使用关于代码和程序操作等通常在测试人员视野之外的信息设计测试。

14.3.2 按照是否需要执行被测程序分类

1. 静态测试

静态测试是指不运行被测程序本身,仅通过分析或检查源程序的语法、结构、过程、接口等来检查程序的正确性,通过对需求规格说明书、软件设计说明书、源程序做结构分析、流程图分析、符号执行来发现错误。静态测试方法通过对程序静态特性的分析,找出欠缺和可疑之处,例如,不匹配的参数、不适当的循环嵌套和分支嵌套、不允许的递归、未使用过的变量、空指针的引用和可疑的计算等。静态测试结果可用于进一步查错,并为测试用例选取提供指导。

静态测试包括代码检查、静态结构分析、代码质量度量等。它可以由人工进行,充分发挥人的逻辑思维优势,也可以借助软件工具自动进行。代码检查包括代码走查、桌面检查、代码审查等,主要检查代码和设计的一致性,代码对标准的遵循、可读性,代码的逻辑表达的正确性,代码结构的合理性等方面。通过代码检查可以发现违背程序编写标准的问题,程序中不安全、不明确和模糊的部分,找出程序中不可移植部分、违背程序编程风格的问题,包括变量检查、命名和类型审查、程序逻辑审查、程序语法检查和程序结构检查等内容。

在实际使用中,代码检查比动态测试更有效率,能快速找到缺陷,发现30%~70%的逻辑设计和编码缺陷。代码检查看到的是问题本身而非征兆。但是代码检查非常耗费时间,而且代码检查需要知识和经验的积累。代码检查应在编译和动态测试之前进行,在检查前,应准备好需求描述文档、程序设计文档、程序的源代码清单、代码编码标准和代码缺陷检查表等。静态测试具有发现缺陷早、降低返工成本、覆盖重点和发现缺陷的概率高的优点以及耗时长、不能测试依赖和技术能力要求高的缺点。

2. 动态测试

动态测试方法是指通过运行被测程序,检查运行结果与预期结果的差异,并分析运行效率、正确性和健壮性等性能。这种方法由三部分组成:构造测试用例、执行程序、分析程序的输出结果。

动态测试的主要目的是确保软件在安装期间和之后都能正常运行,从而确保稳定的应用程序没有任何重大缺陷。

14.3.3 自动化测试

自动化测试（Automated Testing）指软件测试的自动化，在预设状态下运行应用程序或者系统，预设条件包括正常和异常，最后评估运行结果。自动化测试是将人为驱动的行为转化为机器执行的过程。

常见的自动化测试包括单元自动化测试、接口自动化测试、UI 自动化测试等。

14.4 软件测试类型

14.4.1 按测试对象划分

1. 功能测试

功能测试（Functional Testing）根据产品特性、操作描述和用户场景来测试产品的特征和可操作行为，以确定其满足设计需求。功能测试软件用来验证应用程序或网站是否能够为目标用户正常工作。选用合适的平台、浏览器和测试脚本来确保目标用户的体验足够好，就像应用程序是专门为该市场开发的一样。功能测试是根据功能要求对软件进行的测试，确保程序按照预期的方式运行，通过测试系统的所有特性和功能来确保程序与规范的一致性。

功能测试又称为数据驱动测试，只需要考虑要测试的各种功能，不需要考虑整个软件的内部结构和代码。功能测试一般从软件产品的界面、架构出发，根据需求编写测试用例，输入期望结果和实际结果之间的对比，从而提出使产品更加贴近用户使用的要求。

功能测试的 6 个步骤为：

- （1）需求分析人员根据需求对功能进行预期编写。
- （2）测试工程师在需求分析师给出的执行预期的基础上编写测试案例。
- （3）测试工程师根据需求分析师给出的执行预期，准备与预期结果相关的必要数据，包括测试所需数据。
- （4）测试工程师在测试用例的基础上执行测试用例。
- （5）测试工程师对实际结果与预期结果是否一致，并给出报告报告。
- （6）需求分析人员根据测试工程师提供的报告来判断一个应用是否满足了用户的需要。

2. 性能测试

性能测试（Performance Testing）是指测试在不同的情况下软件响应的时间，检查系统是否满足需求规格说明书中规定的性能。性能测试通过自动化测试工具模拟多种正常、峰值以及异常负载条件来对系统的各项性能指标进行测试。

性能测试又包括负载测试（Load Testing）、压力测试（Stress Testing）、并发测试、容量测试（Volume Testing）、可靠性测试等。注意，负载测试和压力测试可以结合进行。

（1）负载测试：是在系统性能指标已经有一项或几项达到了瓶颈的情况下，让它继续运行一段时间，测试系统运行过程中各项性能指标。这是一种在已经达到了最大负载（压力）时进行的测试。

(2) 压力测试: 从字面意思来讲, 就是给某物一定的压力, 比如用手按压桌子等。压力测试就是给被测系统不断地加压(负载), 测试系统的各项性能指标, 直到系统的某些性能指标达到了瓶颈为止, 比如 CPU 的使用率达到了 100%。这个不断给系统施加压力的过程就称为压力测试。

(3) 并发测试: 是一个负载测试和压力测试结合的过程, 即逐渐增加负载, 直到系统的瓶颈或者不能接受的性能点, 通过综合分析交易执行指标和资源监控指标来确定系统并发性能的过程。

(4) 容量测试: 容量可以看作是系统性能指标中一个特定环境下的特定性能指标, 即设定的界限或极限值。容量测试的目的是通过测试, 预先分析出反映软件系统应用特征的某项指标的极限值(如最大并发用户数、数据库记录数等), 系统在其极限值状态下没有出现任何软件故障或还能保持主要功能正常运行。

(5) 可靠性测试: 是指为了满足软件的可靠性要求而进行的一系列设计、分析、测试等工作。其中, 确定软件可靠性要求是软件可靠性工程中要解决的首要问题。软件可靠性要求可以包括定性及定量要求。软件可靠性测试是在软件生存周期的系统测试阶段提高软件可靠性水平的有效途径。各种测试方法、测试技术都能发现软件中残存的导致软件失效的缺陷, 排除这些缺陷后, 一般来讲一定会实现软件可靠性的增长, 但是排除这些缺陷对可靠性的提高的作用却是不一样的。其中, 软件可靠性测试能最有效地发现对可靠性影响大的缺陷, 因此可以有效地提高软件的可靠性水平。软件可靠性测试也是评估软件可靠性水平、验证软件产品是否达到软件可靠性要求的重要且有效的途径。

3. 安全测试

安全测试是指有关验证应用程序的安全等级和识别潜在安全性缺陷的过程, 主要是测试应用程序层的安全, 包括以下两个层面。

1) 应用程序本身的安全性

一般来说, 应用程序的安全问题主要是由软件漏洞导致的, 这些漏洞可以是设计上的缺陷或是编程上的问题, 甚至是企业开发人员预留的后门。

2) 应用程序的数据安全

应用程序的数据安全包括数据存储安全和数据传输安全两个方面。

一般来说, 对安全性要求不高的软件, 其安全性测试可以混在单元测试、集成测试、系统测试里一起做。但对安全性有较高需求的软件, 则必须做专门的安全测试, 以便在软件出现问题之前预防并识别软件的安全问题。安全测试的主要目的是查找软件自身程序设计中存在的安全隐患, 并检查应用程序对非法侵入的防范能力, 根据安全指标的不同, 测试策略也有所不同。

软件系统安全测试是用来验证集成在软件内的保护机制是否能够在实际中保护系统免受非法侵入。通俗地说, 软件系统的安全必须能够经受住正面的攻击, 同时它也必须能够经受住侧面的和背后的攻击, 才能保证软件的安全运行。所以对于企业来说, 对软件进行安全测试, 可以有效规避企业的安全风险, 保证企业立于不败之地。

安全测试是一个相对独立的领域, 需要更多的专业知识, 例如, 对 Web 的安全测试需要熟

悉各种网络协议、防火墙、CDN、各种操作系统的漏洞、路由器等相关知识。安全测试主要包括渗透测试、流量攻击、SQL 注入、跨域攻击等方面的测试。

4. 兼容性测试

兼容性测试（Compatibility Testing）主要是测试软件之间能否很好地运作，会不会相互影响，软件和硬件之间能否协调发挥出很好的效率来工作，会不会相互影响导致系统崩溃。

兼容性测试的核心内容包括：

- (1) 测试软件能否在不同的操作系统平台上兼容。
- (2) 测试软件是否能在同一操作系统平台的不同版本上兼容。
- (3) 软件本身能否向前或向后兼容。
- (4) 测试软件能否与其他相关软件兼容。
- (5) 数据兼容性测试，主要指数据能否共享。

最常见的兼容性测试是浏览器的兼容性测试，不同浏览器在 .css、.js 解析上的不同会导致页面显示的不同。

5. 界面测试

界面测试（UI Testing）主要测试用户界面的功能模块的布局是否合理、整体风格是否一致、各个控件的放置位置是否符合客户使用习惯，此外还要测试界面操作的便捷性、导航是否简单易懂、页面元素的可用性、界面中的文字是否正确、命名是否统一、页面是否美观、文字与图片组合是否完美等，即检查界面是否美观，从风格是否统一、布局是否合理、配色是否合适等方面进行测试。

6. 易用性测试

易用性（Useability）是交互的适应性、功能性和有效性的集中体现，又叫用户体验测试。

易用性测试流程如下：

- (1) 根据项目需求，结合通用的易用性要求，裁剪出适合项目的易用性检查清单，也就是易用性测试用例。
- (2) 项目组成员对易用性达成一致意见，以保证软件的易用性水平。
- (3) 按“软件易用性检查表”逐一进行排查，按正常 bug 操作流程进行操作管理，尽早提交相关负责人。
- (4) 对照标准，对所测软件的易用性进行合理的评估，并记录到软件测试报告中。

7. 稳定性测试

稳定性测试（Endurance Testing）是通过长时间运行软件来测试软件是否会出现问题。软件稳定性测试的常用测试点包括：

- (1) 对软件进行多次测试，长时间运行，观察软件和系统是否正常运行。
- (2) 长时间对软件进行开启关闭，观察软件和系统是否正常进行。
- (3) 软件长时间执行某个业务后切换到别的不同的业务操作是否受影响。
- (4) 软件长时间开启但是不执行任何操作，然后检查能否正常执行业务操作。

(5) 软件长时间对日常的用户数进行操作运行, 观察系统内存占用率是否越来越大, 可用内存是否减少, 内存是否溢出, 饱和运算内存是否占用过大、是否溢出等。

14.4.2 按测试阶段划分

1. 单元测试

单元测试 (Unit Testing) 是指对软件中的最小可测试单元进行检查和验证。对于单元测试中的单元, 要根据实际情况去判定其具体含义。例如, C 语言中单元指一个函数, Java 中单元指一个类, 图形化的软件中单元可以指一个窗口或菜单等。总的来说, 单元就是人为规定的最小的被测功能模块。单元测试是在软件开发过程中要进行的最低级别的测试活动, 软件的独立单元将在与程序的其他部分相隔离的情况下进行测试。

单元测试以白盒技术为主, 黑盒技术为辅。单元测试一般由开发人员本身完成, 并在开发组组长的监督下进行。在进行单元测试时使用的数据, 通常不使用真实数据, 当被测单元要求操纵大量数据, 并且有很多单元都有这种需求时, 可以考虑使用正式数据的一个较小的有代表性的样本。

单元测试的相关规定如下:

- 测试阶段: 编码后或者编码前。
- 测试对象: 模块。
- 测试人员: 白盒测试工程师或开发人员。
- 测试依据: 代码、注释和详细文档。
- 测试方法: 白盒测试。
- 测试内容: 模块接口测试、局部数据测试、路径测试、错误处理测试、边界测试。

2. 集成测试

集成测试 (Integration Testing) 是指将软件集成起来后进行测试。集成测试又叫子系统测试、组装测试、部件测试等。集成测试主要是针对软件高层设计进行测试, 一般来说是以模块和子系统为单位进行测试。

集成测试包含的层次:

- (1) 模块内的集成, 主要是测试模块内各个接口间的交互集成关系。
- (2) 子系统内的集成, 测试子系统内各个模块间的交互关系。
- (3) 系统集成, 测试系统内各个子系统和模块间的集成关系。

集成测试的相关规定如下:

- 测试阶段: 一般是单元测试之后。
- 测试对象: 模块间的接口。
- 测试人员: 白盒测试工程师或开发工程师。
- 测试依据: 单元测试的文档和概要设计文档。
- 测试方法: 黑盒测试、白盒测试以及灰盒测试。
- 测试内容: 模块之间数据传输、模块之间功能冲突、模块组装功能的正确性、全局数据结构、单模块缺陷对系统的影响。

3. 系统测试

系统测试（System Testing）是对整个系统的测试，将硬件、软件、操作人员看作一个整体，检验它是否有不符合系统说明书的地方。这种测试可以发现系统分析和设计中的错误，如安全测试是测试安全措施是否完善，能不能保证系统不受非法侵入；再如，压力测试是测试系统在正常数据量以及超负荷量（如多个用户同时存取）等情况下是否还能正常地工作。

系统测试是将经过集成测试的软件，作为计算机系统的一个部分，与系统中其他部分结合起来，在实际运行环境下对计算机系统进行的一系列严格有效的测试，以发现软件潜在的问题，保证系统的正常运行。其目的是验证最终软件系统是否满足用户规定的需求。

比较常见的、典型的系统测试包括恢复测试、安全测试、压力测试等。

（1）恢复测试。恢复测试作为一种系统测试，主要关注导致软件运行失败的各种条件，并验证其恢复过程能否正确执行。在特定情况下，系统需具备容错能力。另外，系统失效必须在规定时间段内被更正，否则将会导致严重的经济损失。

（2）安全测试。安全测试用来验证系统内部的保护机制，以防止非法侵入。在安全测试中，测试人员扮演试图侵入系统的角色，采用各种办法试图突破防线。因此系统安全设计的准则是要想方设法使侵入系统所需的代价更加昂贵。

（3）压力测试。压力测试是指在正常资源下使用异常的访问量、频率或数据量来执行系统。在压力测试中可执行以下测试：

- ①如果平均中断数量是每秒一到两次，那么可设计特殊的测试用例，比如产生每秒十次中断。
- ②输入数据量增加一个量级，确定输入功能将如何响应。
- ③在虚拟操作系统下，产生需要最大内存量或其他资源的测试用例，或产生需要过量磁盘存储的数据。

系统测试包括对功能、性能以及软件所运行的软硬件环境进行测试，方法包括回归测试和冒烟测试。这是检验系统是否确实能提供系统方案说明书中制定功能的有效方法。

系统测试的相关规定如下：

- 测试阶段：集成测试阶段之后。
- 测试对象：整个系统（软件、硬件）。
- 测试人员：黑盒测试工程师。
- 测试依据：需求规格说明文档。
- 测试方法：黑盒测试。
- 测试内容：功能、界面、可靠性、易用性、性能、兼容性、安全性等。
- 测试策略：性能测试、负载测试、强度测试、易用性测试、安全测试、配置测试、安装测试、文档测试、故障恢复测试、用户界面测试、恢复测试、分布测试、可用性测试。

4. 验收测试

验收测试（Acceptance Testing）是部署软件之前的最后一个测试操作，是在软件产品完成了单元测试、集成测试和系统测试之后、产品发布之前所进行的软件测试活动。它是技术测试

的最后一个阶段，也称为交付测试。验收测试的目的是确保软件准备就绪，并且可以让最终用户将其用于执行软件的既定功能和任务。

验收测试是向未来的用户表明系统能够像预定要求那样工作。经集成测试后，已经按照设计把所有的模块组装成一个完整的软件系统，接口错误也已经基本排除了，接着就应该进一步验证软件的有效性，这就是验收测试的任务，即软件的功能和性能如同用户所合理期待的那样。

实施验收测试的常用策略有三种，分别是正式验收，非正式验收或 Alpha 测试，以及 Beta 测试。

(1) 正式验收测试：是一项管理严格的过程，它通常是系统测试的延续。计划和设计这些测试的周密和详细程度不亚于系统测试。选择的测试用例应该是系统测试中所执行测试用例的子集。

在某些组织中，开发组织（或其独立的测试小组）与最终用户组织的代表一起执行验收测试。在其他组织中，验收测试则完全由最终用户组织执行，或者由最终用户组织选择人员组成一个客观公正的小组来执行。

(2) 非正式验收：在非正式验收测试中，执行测试过程的限定不像正式验收测试中那样严格。在此测试中，确定并记录要研究的功能和业务任务，但没有可以遵循的特定测试用例。测试内容由各测试员决定。这种验收测试方法没有正式验收测试那样组织有序，而且更为主观。

(3) Beta 测试：相比于以上两种验收测试策略，Beta 测试需要的控制是最少的。在 Beta 测试中，采用的细节多少、数据和方法完全由各测试员决定。各测试员负责创建自己的环境、选择数据，并决定要研究的功能、特性或任务。各测试员负责确定自己对于系统当前状态的接受标准。Beta 测试由最终用户实施，通常开发（或其他非最终用户）组织对其的管理很少或不进行管理。Beta 测试是所有验收测试策略中最主观的。

Beta 测试是软件的多个用户在一个或多个用户的实际使用环境下进行的测试。开发者通常不在测试现场，Beta 测试不能由程序员或测试员完成。

验收测试确保软件准备就绪，按照项目合同、任务书、双方约定的验收依据文档，向软件购买者展示该软件系统满足原始需求。

验收测试的相关规定如下：

- 测试阶段：系统测试通过后。
- 测试对象：整个系统（包括软件、硬件）。
- 测试人员：主要是最终用户或者需求方。
- 测试依据：用户需求、验收标准。
- 测试方法：黑盒测试。
- 测试内容：同系统测试（功能、各类文档等）。

14.4.3 按被测软件划分

1. App 测试

App 测试是指对移动应用进行的测试，包括对移动应用的业务、兼容性、健壮性等一系列需求的验证。

App 测试的主要测试内容包括：逻辑测试、功能 /bug 测试、界面测试、兼容性测试、漏洞检查、压力测试、中断测试、弱网测试。

2. Web 测试

Web 测试是软件测试的一部分，是针对 Web 应用的一类测试。由于 Web 应用与用户直接相关，又通常需要承受长时间的大量操作，因此 Web 项目的功能和性能都必须经过可靠的验证。通过测试可以尽可能地多发现浏览器端和服务器端程序中的错误并及时加以修正，以保证应用的质量。由于 Web 具有分布、异构、并发和平台无关的特性，因而它的测试要比普通程序复杂得多。

3. 物联网测试

物联网（Internet of Things, IoT）测试是一种检查物联网设备的测试。现在，人们越来越需要更好、更快的服务，例如多重设备访问，创建、使用和共享数据等，以提供更大的洞察力和控制力。因此，物联网测试非常重要。

物联网测试大致围绕安全、分析、设备、网络、处理器、操作系统、平台和标准进行。

4. 车联网测试

车联网（Vehicle-to-Everything, V2X）是以车辆为主体，依靠通信网络互连实现车与车（V2V）、车与人（V2P）、车与网（V2N）、车与基础设施（V2I）的互联互通、信息共享，进而达到保障交通安全、提高驾驶体验、拓展智能服务等目标。

车联网中，车辆将与其他车辆以及路边基础设施进行通信，共享位置、速度、加速度，甚至防抱死制动系统激活信息。

5. 大数据测试

大数据测试通常是指对采用大数据技术的系统或应用的测试。大数据测试可以分为两个维度，一个维度是数据测试，另一个维度是大数据系统测试和大数据应用产品测试。

大数据系统测试主要包括功能、基准、安全和可靠性测试。功能测试主要是对数据的采集和传输、数据的存储和管理、数据计算、数据的查询和分析，以及数据的可视化等功能的测试。基准测试主要用于对比和评估大数据框架组件的性能指标等。

6. AI 测试

AI 测试是对基于 AI 的应用系统的测试。与任何系统一样，基于 AI 的系统具有功能性和非功能性需求。某些传统的测试方法也会在 AI 软件测试中用到，例如在代码级的单元测试，以及 AI 系统的一些性能测试、可伸缩性测试、安全性测试等。

由于 AI 系统与传统系统所采用的技术有很大不同，如机器学习、逻辑程序、规则引擎、演绎分类器、神经网络等，再加上 AI 系统的特点是不确定性和概率性、对大数据具有依赖性、随机性的输入 / 输出、难以预测所有应用场景、需要从过去的行为中不断自我学习等，因此传统的方法和测试工具将不能满足 AI 测试的需要。

AI 测试要面临很多挑战，例如，缺乏涵盖基于完整性的 AI 系统开发标准、缺乏高度集成

的系统性测试、对 AI 交互系统如何导致故障的理解不深等。

AI 系统的一些独特性（如算法、模型）有别于传统的测试需求，需要不同的测试，例如，算法的健壮性、模型评估、模型安全等测试。基于 AI 的系统具有一些独特的质量特征，例如适应性、自主性、演化、灵活性、偏差、性能、透明度、复杂性和不确定性，这些质量特征可用作测试检查项的基础，以确定需要测试的范围和类型。

7. 小程序测试

小程序测试与 App 测试的区别包括：①获取渠道不同，小程序测试一般可以直接从微信端打开，而 App 测试需要下载安装；②测试时间长短不一样，小程序测试的开发周期和平台审核周期都比较短，而 App 测试内容较多，需要的测试时间更长。

从测试的内容及测试要点来看，涉及小程序的测试内容主要有以下几个方面：

- (1) 权限测试：包括授权登录小程序测试、能够查看的数据及用户权限等。
- (2) UI 测试：包括小程序页面展示是否准确、文字图片内容是否完善、界面的交互效果等。
- (3) 功能测试：根据小程序的各个功能模块进行测试，并对各个业务流程进行测试。
- (4) 兼容性测试：对不同操作系统测试、分享测试等。例如，目前很多小程序都只能生成分享程序图片的方式，这就需要用户做好不同屏幕的兼容适配工作。
- (5) 其他测试：例如网络测试、接口测试等。

14.4.4 其他分类

1. 回归测试

回归测试是指修改了旧代码后重新进行测试，以确认修改没有引入新的错误或导致其他代码产生错误。自动回归测试将大幅降低系统测试、维护升级等阶段的成本。

回归测试作为软件生命周期的一个组成部分，在整个软件测试过程中占有很大的比重，软件开发各个阶段都会进行多次回归测试。在渐进和快速迭代开发中，新版本的连续发布使回归测试进行得更加频繁，而在极限编程方法中，更是要求每天都进行若干次回归测试。因此，通过选择正确的回归测试策略来改进回归测试的效率和有效性是很有意义的。

回归测试需要时间、经费和人力来计划、实施和管理。为了在给定的预算和进度下，尽可能有效率和有效地进行回归测试，需要对测试用例库进行维护并依据一定的策略选择相应的回归测试包。

选择回归测试策略应该兼顾效率和有效性两个方面。常用的回归测试策略包括以下几种。

1) 再测试全部用例

选择基线测试用例库中的全部测试用例组成回归测试包，这是一种比较稳妥的测试方法，再测试全部用例具有最低的遗漏回归错误的风险，但测试成本最高。全部再测试几乎可以应用于任何情况，基本上不需要进行分析和重新开发，但是，随着开发工作的进展，测试用例不断增多，重复原先所有的测试将带来很大的工作量，往往超出了项目的预算和进度。

2) 基于风险选择测试

可以基于一定的风险标准来从基线测试用例库中选择回归测试包。首先运行最重要的、关

键的和可疑的测试，而跳过那些非关键的、优先级别低的或者高稳定的测试用例，这些用例即便可能测试到缺陷，其严重性也仅有三级或四级。一般而言，测试顺序是从主要特征到次要特征。

3) 基于操作剖面选择测试

如果基线测试用例库的测试用例是基于软件操作剖面开发的，则测试用例的分布情况反映了系统的实际使用情况。回归测试所使用的测试用例个数可以由测试预算确定，回归测试可以优先选择那些针对最重要或最频繁使用功能的测试用例，释放和缓解最高级别的风险，有助于尽早发现那些对可靠性有最大影响的故障。这种方法可以在一个给定的预算下最有效地提高系统可靠性，但实施起来有一定的难度。

4) 再测试修改的部分

当测试者对修改的局部化有足够的信心时，可以通过相依性分析识别软件的修改情况并分析修改产生的影响，将回归测试局限于被改变的模块和它的接口上。通常，一个回归错误一定涉及一个新的、修改的或删除的代码段。在允许的条件下，回归测试应尽可能地覆盖受到影响的部分。

再测试全部用例的策略是最安全的策略，但已经运行过许多次的回归测试不太可能揭示新的错误，而且很多时候，由于时间、人员、设备和经费的原因，不允许选择再测试全部用例的回归测试策略，此时，可以选择适当的策略进行缩减的回归测试。

在实际工作中，回归测试需要反复进行，当测试者一次又一次地完成相同的测试时，这些回归测试将变得非常令人厌烦，而在大多数回归测试需要手工完成的时候尤其如此，因此，需要通过自动测试来实现重复的和一致的回归测试。通过测试自动化可以提高回归测试效率。为了支持多种回归测试策略，自动测试工具应该是通用和灵活的，以便满足达到不同回归测试目标的要求。例如，安排新的测试者完成手工回归测试，分配更有经验的测试者开发新的测试用例，编写和调试自动测试脚本，做一些探索性的或 Ad-Hoc 测试（随机测试）。还可以在不影响测试目标的情况下，鼓励测试者创造性地执行测试用例，变化的输入、按键和配置有助于激励测试者揭示新的错误。

2. 冒烟测试

冒烟测试这一术语源自硬件行业，它是对一个硬件或硬件组件进行更改或修复后，直接给设备加电，如果没有冒烟，则该组件就通过了测试。在软件中，冒烟测试这一术语描述的是在将代码更改嵌入产品的源树之前对这些更改进行验证的过程。在检查了代码后，冒烟测试是确定和修复软件缺陷的最经济有效的方法。冒烟测试设计用于确认代码中的更改会按预期运行，且不会破坏整个版本的稳定性。

冒烟测试的对象是每一个新编译的需要正式测试的软件版本。通过冒烟测试，在软件代码正式编译并交付测试之前，先尽量消除其表面的错误，减少后期测试的负担。冒烟测试的执行人是版本编译人员。因此可以说，冒烟测试是预测试。在实际的软件测试工作中，冒烟测试在软件研发的不同阶段有所不同，大体可以分为三类：

(1) 形成集成测试版本以前：验证各个单元能够成功执行，并保证测试版本能够顺利集成。

(2) 形成集成测试版本：以保证新的或者更改过的代码不破坏集成版本的完成性和稳定性。

(3) 后期预测测试缺陷的修正：针对每个缺陷所做的缺陷修正都要先在干净的链接环境中进行冒烟测试，测试通过后才能更新相关软件版本。

冒烟测试在软件生命周期中所占的时间比例较低，同时具有注重通过性、轻细节的特点，因此经常被开发、测试人员所忽视。事实上，冒烟测试是软件测试过程中一个不可或缺的节点，一个好的冒烟测试过程，对于软件测试效率的提升具有重要意义。

14.5 软件测试的组织

14.5.1 测试过程

软件测试过程主要包括以下几个阶段：

(1) 项目需求分析阶段：项目需求是整个测试过程的前置条件，也是任何一个测试工作开始前的第一步。所谓项目需求分析，是指测试员分析和确定被测试的项目的测试工作安排，也就是该项目需要测哪些模块，什么样的结果是符合产品需求的预期结果。

(2) 制订测试计划阶段：测试计划制订阶段的主要任务就是编写完整的测试流程安排，测试哪些模块，用什么方法测试（例如，常见的冒烟测试、随机测试、黑盒测试、性能测试等），以及不同测试阶段的测试人员的工作安排、工作目标等。这个阶段对测试主管的整体测试调控能力有很高的要求。

(3) 设计测试文档阶段：这个阶段是整个软件测试的基础，也是核心阶段。这个阶段的主要工作是参考项目的设计原型图、产品的设计需求和详细设计标准等编写测试用例。

(4) 搭建测试环境阶段：不同的产品对测试环境有着不同的要求。能做好软件测试工程师这门工作的一个指标就是能独立搭建项目的测试环境。

(5) 执行测试阶段：这个阶段就开始正式的测试工作了。一般包括单元测试、集成测试、系统测试、验收测试以及回归测试等。每个阶段对应了不同的测试具体执行细节。

(6) 缺陷管理：缺陷管理主要是用于收集、管理和沟通项目缺陷的过程，重点考察的是缺陷的识别和缺陷的解决。这个阶段一般会用到一些系统化的缺陷管理工具。

14.5.2 测试管理

1. 配置管理

软件测试配置管理一般应用过程方法和系统方法来建立软件测试管理体系，也就是把软件测试管理作为一个系统，对组成这个系统的各个过程加以识别和管理，以实现设定的系统目标。同时要使这些过程协同作用、互相促进，从而使它们的总体作用大于各个过程之和。软件测试配置管理的主要目标是在设定的条件限制下，尽可能发现和找出软件缺陷。测试配置管理是软件配置管理的子集，作用于测试的各个阶段。其管理对象包括软件测试计划、测试方案（用例）、测试版本、测试工具及环境、测试结果等。

配置管理的目标：

- (1) 控制和审计测试活动的变更;
- (2) 在测试项目的里程碑建立相应的基线;
- (3) 记录和跟踪测试活动变更请求;
- (4) 相应的软件测试活动或产品 (work products) 被标识、控制, 并且是可用的。

配置管理的承诺:

- (1) 每个测试项目的配置管理责任明确;
- (2) 配置管理贯穿项目的整个测试活动;
- (3) 配置管理应用于所有的测试配置项, 包括支持工具;
- (4) 建立配置库和基线库 (baseline);
- (5) 定期评审基线库内容和测试配置项活动。

2. 过程管理

软件测试的过程管理并不仅限于测试阶段, 所有的测试都是基于需求的, 在需求分析阶段, 测试团队要依据需求文档提取测试点。

测试的过程管理的基本内容如下:

(1) 测试项目启动阶段: 首先需要确定项目负责人, 即项目小组组长, 项目组长确定以后, 才可以组建整个测试小组, 配合开发等部门开展工作; 其次要参加有关项目计划、分析和设计的会议, 获得必要的需求分析、系统设计文档, 以及相关产品 / 技术知识的培训和转移。

(2) 测试计划阶段: 确定测试范围、测试策略和方法, 对风险、日程表、资源等进行分析和估计。

(3) 测试设计阶段: 制定测试的技术方案, 设计测试用例, 选择测试工具, 编写测试脚本等。测试用例设计需要做好各项准备再开始进行, 最后还需要其他部门帮忙评审测试用例。

(4) 测试执行阶段: 搭建相关的测试环境, 准备测试数据, 执行测试用例, 对发现的软件缺陷进行报告、分析、跟踪等。测试执行不涉及较高的技术性, 但它是测试的基础, 直接关系到测试的可靠性、客观性和准确性。

(5) 测试结果的审查和分析阶段: 测试执行结束后, 需要对测试结果进行整体或综合的分析, 以确定软件产品质量的当前状态, 为产品的改进或发布提供数据和依据。从管理上讲, 需要组织好测试结果的评审和分析会议, 做好测试报告或质量报告的编写和评审。

3. 缺陷管理

软件缺陷有很多, 从不同的角度可以将缺陷分为不同的种类。

- (1) 按照测试种类可以将软件缺陷分为界面类、功能类、性能类、安全性类、兼容性类等。
- (2) 按照缺陷的严重程度可以将缺陷划分为严重、一般、次要、建议。
- (3) 按照缺陷的优先级不同可以将缺陷划分为立即解决、高优先级、正常排队、低优先级。
- (4) 按照缺陷的发生阶段不同可以将缺陷划分为需求阶段缺陷、构架阶段缺陷、设计阶段缺陷、编码阶段缺陷、测试阶段缺陷。

当测试人员发现软件中存在的缺陷之后, 我们还需要对这些缺陷进行统一管理。

- (1) 确定缺陷优先级: 在评估缺陷的优先级时, 可以从单个或多个维度评估, 通常情况下

可从影响范围和严重级别这两个常用的维度评估。

(2) 及时同步缺陷状态: 按优先级安排好之后就可以制订修复计划并开始修复了。当修复完成时, 要及时将修复信息同步给相关的测试人员、用户, 这一过程可以借助缺陷管理软件来完成。

4. 风险管理

在软件的测试活动过程中, 通过一系列风险控制活动对不确定因素进行及时识别、有效评估、积极应对, 以保证软件测试活动的正常进行, 从而达到价值交付。风险管理的主要内容如下:

(1) 风险识别: 在风险识别阶段, 通过树立团队整体的风险意识, 大家一起讨论制定风险识别维度, 在对应的研发阶段, 利用一定的工具方法进行风险数据收集, 从而进行有效识别。

(2) 风险评估: 通过有效描述风险, 明确风险类别后合理定义风险级别来评估风险, 以确认风险影响以及为后续的风险应对措施提供依据。

(3) 风险应对: 在风险评估定级完成后, 进行对应的风险应对; 在实际的项目运作过程中, 根据不同的场景采用不同的应对措施。

(4) 风险上报: 当超出个人或团队处理能力时, 需要进行风险上报, 一般风险级别为高以上类的风险(比如涉及外部依赖类风险)需要项目协调, 否则版本无法正常集成制作。通过面对面、电话沟通, 或者日报/周报的方式上报反馈。

(5) 风险规避: 主要针对发生的概率较高, 具有严重负面影响的高优先级风险, 例如, 在临近版本发布时, 某些特性由于测试不充分, 波及不明确, 合入后对现有商用版本存在未知隐患, 则通过裁剪、调整特性合入周期来规避版本发布质量风险。

(6) 风险转移: 指应对风险, 把责任外包给第三方, 例如, 在研发过程中, 对于新领域不熟悉, 但第三方存在成熟的技术, 可以通过购买服务、签约合同以及和第三方达成合作, 把风险转移给第三方。

(7) 风险减轻: 指降低风险出现的概率和影响, 例如, 在临近版本发布时, 合入了故障, 但该故障的波及范围较广, 测试又不充分, 经过分析故障影响, 回退合入故障, 来规避影响。

(8) 风险接受: 针对低优先级的风险或者采取其他任何策略已无法应对的风险。

14.6 软件部署

软件部署环节是指将软件项目本身, 包括配置文件、用户手册、帮助文档等进行收集、打包、安装、配置、发布的过程。在信息产业高速发展的时代, 软件部署工作越来越重要。

14.6.1 部署目标

软件部署工作的目标具体如下:

- (1) 保障软件系统的正常运行和功能实现。
- (2) 简化部署的操作过程, 提高执行效率。

- (3) 必须满足软件用户在功能和非功能属性方面的个性化需求。
- (4) 最重要的是要支持软件运行，满足用户需求，使得软件系统能够被直接使用。

14.6.2 部署步骤

1. 环境准备

在目标机器上安装与软件相关的各种依赖项和运行环境，例如操作系统、数据库软件、网络配置等，以保证软件能够正常运行。

2. 软件安装

将软件程序的各个组件文件复制到目标机器指定的位置，并进行一些必要的配置，例如设置文件权限、添加注册表项等，以便软件能够正确地运行。

3. 数据库和文件配置

对于使用数据库和文件存储的软件，需要在目标机器上建立相应的数据库和文件存储位置，并进行配置，以便数据能够正确地存储和读取。

4. 测试和验证

在软件部署完成后，需要进行一系列的测试和验证工作，以确保软件能够正常运行，并发现和修复潜在的问题和漏洞。